

Речь к презентации проекта Awesome Feature Navigation

Проектная презентация

17 мая 2026 г.

Слайд 1. Титульный слайд

Здравствуйте. Я представляю проект `awesome-feature-navigation`. Это Python-библиотека и CLI-инструмент для построения 2D-траектории робота по видео с правой камеры ZED и данным IMU. Важная формулировка: проект решает прикладную задачу восстановления траектории для нашей записи, а не претендует на полноценный visual-inertial SLAM.

Слайд 2. Цель проекта

Цель проекта - получить понятную траекторию движения робота из уже записанных данных. На входе есть видео, IMU CSV, timestamps кадров и Kalibr-калибровки. На выходе проект сохраняет CSV с точками `t, x, y, yaw` и HTML-график, который можно открыть в браузере. Основной сценарий сейчас офлайн: мы анализируем всю запись целиком и можем использовать сглаживание по времени.

Слайд 3. Входные данные

Основной видеофайл ожидается как `data/Right_cam.mp4`. IMU лежит в `data/imu_data.csv`. Для синхронизации с видео используется `data/timestamps2.csv`, где находятся реальные timestamps кадров. Еще два важных файла приходят из Kalibr: `imu-imu_calibration.yaml` с шумами IMU и `camchain-imucam-imu_calibration.yaml` с трансформацией между камерой и IMU, а также `time shift`.

Слайд 4. Типовой запуск

Проект запускается командой `uv run afn-run`. В обычном запуске мы передаем путь к видео, путь к IMU, конфиг `configs/right_camera.yaml` и `output prefix`. После выполнения появляются `outputs/right_trajectory.csv` и `outputs/right_trajectory.html`. Для анализа качества можно добавить `--save-loop-debug`, тогда сохраняются `raw`, `smoothed`, `diagnostics` и `laps outputs`.

Слайд 5. Архитектура

Код разделен по зонам ответственности. `cli.py` собирает конфигурацию и запускает пайплайн. `calibration.py` читает timestamps и Kalibr YAML. `imu_io.py` загружает IMU, применяет временной сдвиг, гетар осей и поворот в camera frame. `line_detection.py` отвечает за поиск цветной линии. Основная логика оценки траектории находится в `trajectory.py`, а сохранение CSV и HTML вынесено в `plotting.py`.

Слайд 6. Конфигурация проекта

Главный конфиг - `configs/right_camera.yaml`. В нем включен `trajectory_mode: auto`, а значит проект сам выбирает подходящий estimator. Для нашей записи установлен приоритет `tape_line`, потому что на видео видна синяя линия. Поэтому `target_color` равен `blue`. Также включены `offline smoothing`, `adaptive confidence`, `variable speed` и `representative lap` для финального замкнутого круга.

Слайд 7. Синхронизация и калибровки

Одна из важных частей проекта - правильно привести время и координаты. Timestamps кадров и IMU переводятся из наносекунд в секунды. Если время выглядит абсолютным, оба потока нормализуются относительно первого видеокadra. Kalibr задает `timeshift_cam_imu` по соглашению `t_imu = t_cam + timeshift_cam_imu`, поэтому IMU сдвигается на минус это значение, чтобы нарезать измерения по времени камеры. Затем accel и gyro поворачиваются из IMU frame в camera frame через `T_cam_imu`.

Слайд 8. Выбор режима траектории

В проекте есть два режима. `tape_line` использует синюю линию в кадре и IMU yaw. Это основной режим для текущего видео. `generic_vio` использует optical flow и IMU yaw, но масштаб в нем относительный, потому что по monocular-видео без depth или stereo нельзя надежно получить абсолютный масштаб. В режиме `auto` проект сначала пробует предпочтительный estimator и проверяет, достаточно ли точек, надежных кадров и spatial span.

Слайд 9. Режим `tape_line`

В `tape_line` каждый кадр уменьшается, переводится в HSV, затем по blue HSV-диапазону строится бинарная маска линии. Из маски оставляется крупнейшая компонента, после чего строится centerline через distance transform. Для кадра сохраняются угол линии, позиция линии внизу кадра, confidence и IMU delta yaw. После этого наблюдения сглаживаются по всей записи, а состояние `x, y, yaw` интегрируется с forward speed, yaw correction и lateral correction.

Слайд 10. Режим `generic_vio`

`generic_vio` нужен как fallback, когда линия плохо видна. Он берет grayscale-кадры, находит feature points, отслеживает их Lucas-Kanade optical flow и отбрасывает нестабильные треки через forward-backward check. Затем через `estimateAffinePartial2D` оценивается 2D-transform. Visual yaw смешивается с IMU yaw: если tracking слабый, вес IMU становится выше.

Слайд 11. Loop closure и финальный круг

Если робот ездит по повторяющейся петле, траектория может дрейфовать. Поэтому проект пытается оценить период круга по compact grayscale descriptors. Полный сглаженный путь остается доступен как `smoothed_traj`. Если круги хорошо согласованы, строится `final_traj` - один замкнутый representative lap. Плохие круги не принимаются: для этого есть проверки по alignment RMSE и projection RMSE.

Слайд 12. Выходные артефакты

Минимальный результат - это `outputs/right_trajectory.csv` и `outputs/right_trajectory.html`. Если включить loop debug, появляются raw, smoothed, diagnostics и laps файлы. Это важно, потому что можно отдельно посмотреть исходную траекторию, сглаженный путь, confidence линии, оценки углов, speed и качество нарезки кругов.

Слайд 13. Проверки качества

В проекте есть несколько уровней проверок. `pytest` покрывает загрузку калибровок, работу с IMU, выбор HSV-диапазонов и offline tape-line логику. `ruff` проверяет стиль, импорты и часть потенциальных ошибок. `myru` проверяет типы. `compileall` дополнительно проверяет синтаксис Python-файлов. В бейдже сейчас указан coverage 34.8%, то есть тесты есть, но зону покрытия еще можно расширять.

Слайд 14. Ограничения и допущения

У проекта есть честные ограничения. Видео может не лежать в Git из-за размера, его нужно положить в data вручную. Режим `tape_line` зависит от того, насколько хорошо видна синяя линия и насколько подходят HSV-настройки. Метрический масштаб задается `forward_speed_mps`, поэтому ошибка фактической скорости влияет на масштаб. Translation из `T_cam_imu` загружается, но текущая 2D-оценка позы его не использует.

Слайд 15. Итог

Итог такой: проект превращает видео, IMU и Kalibr-калибровки в воспроизводимый CLI-пайплайн. Основной сценарий - офлайн-навигация по синей линии с IMU yaw. Система сохраняет не только финальный результат, но и отладочные траектории и диагностику качества. За счет режима `auto` и quality gate для кругов проект снижает риск принять плохую оценку траектории за финальную.